

Flexible and Configurable Software Architectures for Data-Intensive Ontology-Based Information Systems

An extended English summary

Michael Wessel

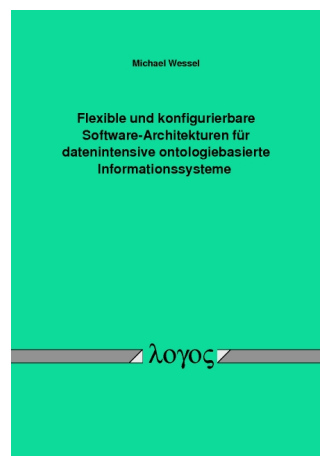
Dissertation submitted 2005, defended 14 April 2008

Technische Universität Hamburg-Harburg

Supervisor: Prof. Dr. Ralf Möller, Institute for Software Systems (STS)

Logos Verlag Berlin, ISBN 978-3-8325-2162-2

This summary: September 24, 2026



This is an extended English summary of a 586-page German dissertation. It follows the thesis chapter by chapter, at enough depth that a reader can understand what was built, why, and what the evaluation established, without reading the German. It closes with a map of which chapters are already available in English as journal and workshop papers, and which are not.

The thesis asks how to build the software substrate for *ontology-based* information systems: systems whose behaviour is driven by a formal ontology

rather than a fixed database schema. It answers by designing, implementing and evaluating a Common Lisp framework across two application domains — a deductive geographic information system and Semantic Web retrieval — and it argues throughout that such a framework should cover *regions* of the design space rather than single points.

Contents

Why this document exists	4
How to read this summary	4
Chapter 1: three problems, not one	4
Regions, not points	5
The secondary storage problem	5
Chapter 2: foundations and terminology	6
Logic	6
Knowledge representation	7
Description logics	7
Description logics versus the relational model	7
Chapter 3: two guiding domains, and what they demand	8
Domain 1: a deductive geographic information system	8
Domain 2: Semantic Web information retrieval	10
The consolidated requirements	10
Chapter 4: the formal framework	12
The substrate	13
The instantiation zoo	13
The generic substrate query language	13
Inference problems for queries, and the QBox	16
Chapter 5: the software realization	17
Realizing the substrate	17
Multimethods: choosing the reasoner per atom	17
The query answering engine	19
The optimisations, and what each was worth	20
Operational design: lifecycle, concurrency, incrementality	21
Four query languages on one framework	22
Chapter 6: MiDeLoRa — the unpublished chapter	24
Why build it at all	24
Provers as points in a space	24
An ABox is a substrate; the tableau is the ABox	24
The command history	25

What that unlocks	25
Base functionality, abstractions and rules	26
The rule definition language	27
Instantiations	29
Chapter 7: evaluation	29
The query engine	30
MiDeLoRa	30
The honest negatives	31
Closing claims	31
Where to read more	32
The dated record	32
The code is public	34
Did it deliver on its title?	34
Appendix: what holds up, and what doesn't	36
Holds up	36
Doesn't	36
The thing actually worth noting	37

Why this document exists

The dissertation it summarises is 586 pages long and written in German. For most of the people who might now have a reason to know what is in it — colleagues in industry, researchers working on description logics, graph databases, query engines or reasoning systems — that makes it effectively unreadable. The work is open access and carries a DOI, and is still almost impossible to assess.

This document is therefore a **record**: what the work set out to do, what was actually built, what was measured, and what it achieved. It is intended to give a reader enough context to judge the work and the background it reflects, without reading the German and without taking anything on trust.

That last point governs how it is written. Claims are tied to things a reader can check — measurements from the evaluation chapter with their figures, class and method names from the published sources, dates from the bibliography. Where a technique disappointed, the measured factor is given. Where something was designed but not implemented, that is said. The companion errata lists defects found on re-reading, including one that turned out to be a defect in the specification rather than the code.

A summary that reported only successes would be less useful for its purpose, not more. The point is to let someone form their own view.

How to read this summary

The thesis divides into three movements. Chapters 1–2 set the problem and the vocabulary. Chapter 3 derives requirements from two concrete application domains. Chapters 4–6 build the answer in three layers — a data model, a query answering framework, and a reasoner construction kit — and Chapter 7 evaluates it.

The summary below follows that order. Coverage is proportional to the thesis except in one place: Chapter 6 is treated more fully than its length alone would justify, because it is the only chapter never published in any form.

One structural point is worth having in advance, because everything else hangs from it. The three layers are not independent modules that happen to be described together. They share a single data model by design, and the thesis's central engineering argument is that they must: a representation layer, a query engine and a reasoner that each carry their own internal model will spend their time translating between models instead of doing work.

Chapter 1: three problems, not one

Building an ontology-based information system forces three distinct problems at once, and the thesis argues they cannot usefully be solved in isolation:

1. **The representation problem** — how are the data represented, and in what data model?
2. **The query answering problem** — how are they retrieved again?
3. **The inference problem** — how are the deductions that (2) already requires actually carried out?

Conventional information systems avoid all three by fixing a schema and making the Closed World and Closed Domain Assumptions: absence of data means negative information, and every relevant object is explicitly modelled. Ontology languages are more expressive than schema languages, so an ontology-based system can represent implicit and indefinite information — and consequently must admit several possible worlds rather than one. That is where the added value comes from, and also where the cost comes from: inference becomes necessary to make implicit domain relationships usable.

Regions, not points

The organising claim is that a framework should cover **regions of the design space, not single points**. Committing a framework to one W3C standard — the common move at the time, and what JENA did with RDFS and OWL — fixes it at a point and forfeits the domains that need something else. The thesis declines to do this, which is why it ends up defining its own data model rather than adopting RDF/RDFS.

The secondary storage problem

The central engineering choice is framed through what the thesis calls the *secondary storage problem*: ontology-based query answering over data too large for main memory. The obvious approach — attach a relational or RDF store through a standard interface — fails, because when expressive ontology languages are used the database cannot compute the answers. It can only supply *candidates*, which must then be loaded into the application layer for inference. To stay complete, the candidate set must always be a superset of the true answers; in the worst case that means loading the entire database.

Three architectures are compared.

Architecture	Inference lives	Cost
Layered	outside the store; the DB	heavy cross-layer traffic; candidate sets must be supersets; DB indexes are “insufficiently informed”
	returns candidates	
Semi-integrated	inside the IS core	same model discontinuity, better locality
Integrated	database functionality	very high implementation effort; no model discontinuity
	moved <i>into</i> the reasoner	

Most work of the era took the layered route, reducing ontology expressivity until the database’s own query engine sufficed — the QuOnto system is cited as an example, using query rewriting to expand ontology vocabulary down to base relations, after which SQL suffices. The Instance Store took the dual route, materialising all derivable facts in advance, at the price of supporting only unary relations.

This thesis takes the opposite route: integrating database functionality into the reasoner. It argues that **high cohesion**, not the usually-sought loose coupling, is right when two components must share index structures — and that the criterion for cohesion is not whether calls are local or remote, but whether the coupled subsystems share a data model.

Chapter 2: foundations and terminology

Chapter 2 (29,100 words) exists for two stated reasons: to make the thesis self-contained, and to fix the author’s terminology. It is careful, textbook-grade exposition, and a reader who knows description logics can skip it — the thesis says so explicitly. It is summarised here because the later chapters use its notation without reintroducing it.

Logic

First-order and propositional logic are introduced to the depth the thesis needs: syntax, semantics, relational structures, models and the model relation, satisfiability, validity, entailment and equivalence, the deduction theorem, compactness, and the undecidability of first-order logic against the decidability of propositional logic. Herbrand interpretations and Herbrand models are defined, as are theories, calculi, and soundness and completeness.

Two less standard definitions matter later: *comprehension* and *constant comprehension*, which give the thesis its way of talking about first-order formulae as set descriptions — the bridge it uses to explain both description logic concepts and query atoms.

Knowledge representation

The chapter then fixes the vocabulary the rest of the thesis leans on: models and representations; the distinction between data, information and knowledge; database systems versus information systems versus knowledge-based systems; rational agents and the architecture of knowledge-based systems; conceptualizations as sets of formulae; relevant notions from knowledge organisation, including thesauri; and knowledge representation languages.

The definition of an ontology follows Gruber — “an ontology is a specification of a conceptualization” — with the stricter variant, a *shared* conceptualization, noted because sharing is what makes ontologies useful to multiple agents. Using an ontology means entering an *ontological commitment*. Only *domain* ontologies are considered; upper ontologies such as CYC are noted and set aside.

Description logics

The description logic material is the operative part of the chapter. After a historical sketch, the base logic $\mathcal{ALC}$ is defined: concept language, semantics, semantic properties, and then knowledge bases. A knowledge base is a pair $\langle \mathcal{A}, \mathcal{T} \rangle$. The TBox is a finite set of axioms $C \sqsubseteq D$ and $C \equiv D$ — general concept inclusions are allowed — and the ABox holds assertions about individuals.

The **standard inference problems** are then defined along with the problem reductions between them, and presented as the API of a description logic system: concept satisfiability, subsumption, ABox consistency, instance retrieval, and so on. This API framing matters, because Chapters 5 and 6 both treat these services as things to be implemented, selected and swapped.

More expressive DLs follow: the DL naming scheme, concept-forming operators, axioms for role properties and complex roles, and **concrete domains** with their admissibility condition — the mechanism by which DLs reach datatypes and numeric constraints, which becomes important for both guiding domains. Complexity results are surveyed, and the relationship to modal logics is drawn via Kripke semantics.

A worked example knowledge base runs through the standard inferences and is reused later in the thesis. Tableau calculi are introduced as the dominant approach for deciding satisfiability, with a full $\mathcal{ALC}$ tableau calculus given and its termination, soundness and completeness discussed. Structural (subsumption) algorithms are covered for contrast. Description logic systems are then described as engineered artefacts, with RacerPro as the running example.

Description logics versus the relational model

The chapter closes with a comparison the thesis needs repeatedly: the relational model is characterised, and the differences from DLs are made explicit — arity (DLs are essentially unary and binary), the closed versus open world assumption, unique

names, and completeness of answers. From this the notion of **ontology-based query answering** is introduced, together with the value-added services it enables: answers that include entailed facts, queries formulated in domain vocabulary, and query-level checks such as consistency and containment.

Chapter 3: two guiding domains, and what they demand

Chapter 3 derives the requirements. Two domains are analysed in depth; they are chosen to pull in opposite directions, so that a framework serving both is demonstrably covering a region rather than a point.

Domain 1: a deductive geographic information system

The first domain uses a real commercial dataset. The **DISK** (*Digitale Stadtkarte*) is a digital city map of the Hamburg district of Öjendorf, sold by the Landesbetrieb Geoinformation und Vermessung Hamburg. It is cartographically generalised vector data — less precise than cadastral data — whose themes are numeric object keys from the Hamburg object key catalogue: residential area, public building, arable land, underground station. The thesis chooses it precisely for that *thematic richness*.

Geo-objects are analysed along three axes: *spatial* aspects, handled qualitatively through the **Region Connection Calculus (RCC)**; *structural* aspects; and *thematic* aspects. A DL-inspired description framework is then laid out in three components — intensional, extensional, and query — and the value-added services of an ontology-based GIS are identified.

The goal is queries mixing theme and space: *a park containing a pond, a school bordering a main road*. If roles exist for the RCC base relations, such a concept is directly expressible:

$$park_with_pond \equiv park \sqcap \exists PPI.pond$$

This motivates a family of description logics with RCC relations, the $\mathcal{ALC}_{RCC}$ **family**. The context is important: the more general related language $\mathcal{ALCRP}(\mathcal{D})$ had already been shown **undecidable**, so the research question is which spatial-thematic fragments remain decidable.

The domain's conclusion is that three things force *special-purpose* provers, which standard DL systems cannot supply:

1. special DLs need special provers (the $\mathcal{ALC}_{RCC}$ family);
2. special inference problems or domain-specific optimisations need them — for instance, closing an RCC ABox over its domain can be done by adding unqualified number restrictions, or, better, by a specialised $\exists$ -rule, which requires an *open* reasoner;
3. the idea of a *spatial ABox*, where individuals are simultaneously instances of spatial datatypes in a spatial index, needs provers that are aware of that richness.

Still live in 2026. Two developments postdate the thesis. The $\forall$ **PO-free fragment** now has a decision procedure, machine-checked in Lean 4; and the **decidability of $\mathcal{ALCI}_{RCC5}$ remains open**, contingent on an unproven width property. This is the one research thread in the dissertation that is not closed history. See *Where to read more*, below.

Domain 2: Semantic Web information retrieval

The second domain is retrieval over RDF and OWL. The Semantic Web vision and stack are surveyed — RDF and RDF Schema, then OWL — along with the query languages available at the time. The concrete scenario is the **Lehigh University Benchmark (LUBM)**, chosen because its dataset can be generated at arbitrary size, which makes it the right instrument for a scalability argument.

The analysis of deficits is specific. An expressive query language is needed that goes beyond what SPARQL and OWL-QL then offered; multi-user operation, incremental and resource-configurable computation of partial results, and complete, performant answering of the LUBM queries are all on the list. The decision is taken to build an ABox query language as the basis for the Semantic Web retrieval language, and the reasoning for that choice is given.

Critically, the OWL reasoner is to be treated as an **exchangeable component**. Implementing one is non-trivial, so existing technology (RacerPro) should be usable bottom-up; but reasoners like RacerPro were built under different requirements, so special-purpose reasoners may serve better top-down. Supporting both perspectives means the ABox query language must be not only *definable* for arbitrary DLs but *realisable* on arbitrary DL systems.

The consolidated requirements

The chapter’s summary states three requirements that drive everything after it.

A flexible, optionally hybrid, semi-structured graph data model. It must represent ABoxes, RDF graphs, RCC networks, the DISK map objects (instances of spatial datatypes) and the literal data values of an OWL document. It needs a formal semantics, so that query language semantics can be defined over it; first-order logic is chosen as the frame. It must support specialised index structures per instantiation — spatial indexes for the DISK. Nodes must be able to carry *rich internal structure*, not merely symbols. And it must give a uniform abstract view over the differing ABox representations of existing DL systems.

The thesis addresses the obvious objection directly. Why not RDF/RDFS? Because rich representations are not readily expressible in it: describing the geometry of DISK objects through RDF predicates is not sensible — languages such as GeoML exist for that — and it is unclear how an RDF query language would then exploit a spatial index. It also insists on *node* annotations and not only edge annotations, since the

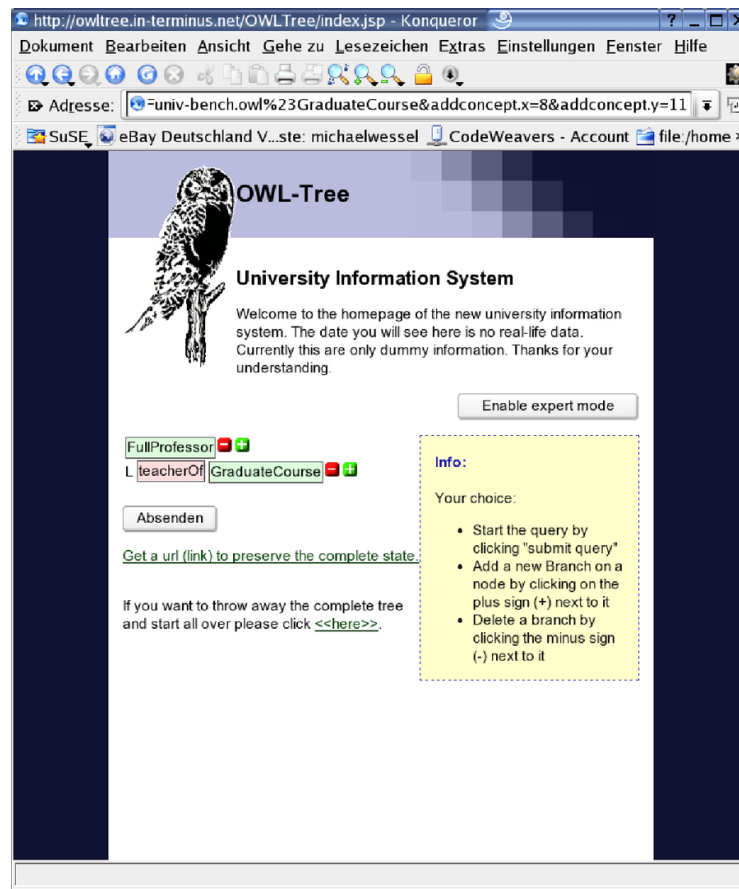


Figure 2: The OWL TREE, a browser front end to the Semantic Web retrieval service, built by **Matthias Flege, Thomas Hirsch, Torben Rebhan and Kamil Sokolski** in a project seminar at TUHH, supervised by the author. Tree-structured conjunctive queries are assembled from concepts and roles — here `FullProfessor`, `teacherOf`, `GraduateCourse` over the LUBM vocabulary — submitted to a `RacerPro` server as `nRQL`, and the result tuples presented for inspection. The team also extended the LUBM generator so that individuals were *real* web resources rather than synthetic `URIRefs` — TUHH staff homepages for `employee`, course pages for `course`, STS publication PDFs for `publication` — making every answer a clickable link to the thing it denoted.

structure of a richly-structured node is not well understood as a set of property-value pairs.

This is where the property-graph bridge is. The requirement is argued here in exactly the vocabulary of property graphs. The thesis asks for annotations on *both* nodes and edges; remarks in passing that node annotations are “attribute-value or **property-value pairs**”; and notes that OWL calls roles “Properties”. It then rejects the pure edge-annotation view — everything decomposed into triples — as inadequate.

What is worth noticing is that it rejects the property-value-pair view *as well*: “the structure of the node itself cannot really be understood as a set of property-value pairs.” A DISK polygon is an instance of a spatial datatype with its own index, not a bag of keys and values. The property graph therefore appears here as a *considered and rejected middle position*, not as an unwitting anticipation — and the substrate passes it in two specific ways:

- node and edge labels are expressions of an arbitrary description language, evaluated by **entailment** rather than retrieved by lookup — so an edge annotation can be a logical statement a reasoner must prove, not a value a store returns;
- nodes may carry internal structure that no key-value map captures, which is what the geometric and spatial substrates need.

For a reader arriving from property graphs, GQL, or RDF-star — all of which exist because annotating and querying edges as first-class objects turned out to matter — this passage, not the formal definition in Chapter 4, is the one to read. It is the same problem statement, reached in 2005 from the description logic side, and then pushed past the answer the industry later settled on.

A framework for defining query languages, matching the data model: optionally hybrid, adaptable per instantiation, formalisable, and efficiently implementable.

Special-purpose provers, which motivate MiDeLoRa — designed to work directly on the same data model, so that a MiDeLoRa ABox is an instance of a specialisation of it. Sharing representation and index structures between the query engine and the provers is what avoids model discontinuities and achieves the cohesion Chapter 1 argued for.

The thesis is explicit that this framework is motivated by **pragmatic and software-engineering reasons rather than theoretical ones**, and argues these deserve at least as much weight as theoretical aspects in a field it characterises as primarily theory-driven. That stance is worth noting, because it explains what the thesis does and does not try to prove.

Chapter 4: the formal framework

Chapter 4 defines the data model and the query language over it, both formally.

The substrate

A **substrate** is a node- and edge-labelled graph

$$(V, E, L_V, L_E, \mathcal{L}_V, \mathcal{L}_E)$$

with $E \subseteq V \times V$, where $L_V : V \rightarrow \mathcal{L}_V$ labels nodes with expressions of a node description language and $L_E : E \rightarrow \mathcal{L}_E$ labels edges likewise.

The semantics is first-order. Nodes correspond to constants; a node annotation is a first-order sentence in prenex normal form whose matrix is in DNF and whose disjuncts reference that node's constant; edge annotations are ground formulae in DNF whose disjuncts are binary ground atoms $R(i, j)$. A ϕ translation function makes this explicit, mirroring the standard translation for DLs given in Chapter 2.

Two further definitions complete the model: the model relation for substrates, and substrates *with background theory*, where the intensional part of a knowledge base is supplied separately as a first-order theory. That separation is the key move — the substrate is extensional, and the ontology sits beside it.

The instantiation zoo

The flexibility claim is made concrete by instantiating $\mathcal{L}_V$ and $\mathcal{L}_E$ and defining the resulting semantics each time: propositional substrates, relational substrates, RCC substrates, geometric substrates with their model theory, qualitative geometric substrates, and qualitative geometric RCC substrates. Inference problems for substrates are then posed, and the substrates needed for the two guiding domains are identified.

A property graph, fused with a reasoner. The *data substrate* — data-node and data-edge, each carrying key-value property maps, so that properties attach to *edges* and not only nodes — is in substance what the industry later called a **property graph**, implemented around 2005 and queryable. The author's own SHACL/nRQL analysis is careful about the claim, and its calibration is the right one to quote: a genuine property graph, but **not the first** — attributed-graph models date to 1984–1994 — and with **no documented influence** on Neo4j, Cypher or GQL. That is “convergent evolution, not lineage.” What was genuinely singular is the combination: **a queryable property graph fused with a tableau DL/OWL reasoner**, so that query answers include entailed facts rather than only asserted ones.

The generic substrate query language

Queries are built from three kinds of atom over a set of objects $O_S = V \cup Vars$, partitioned into individuals and variables:

- a **node atom** (a, v) with $a \in O_S$ and v a node query predicate from a node query language $\mathcal{L}_{QV}$, where $\Phi(v)[x]$ is a first-order formula with one free variable. $\top$ and $\perp$ are required to be present;

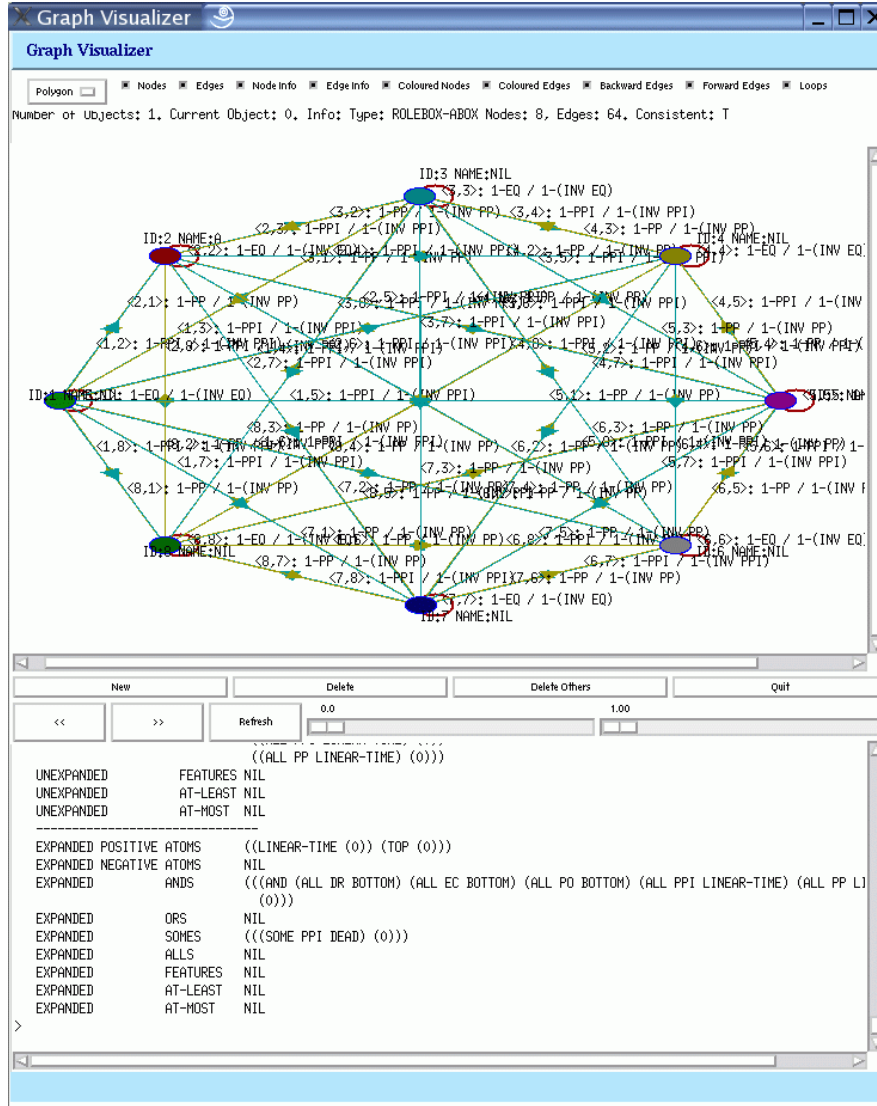


Figure 3: The substrate browser on an RCC substrate — here an RBox ABox of 8 nodes and 64 edges, reported consistent. Every edge carries the qualitative spatial relation holding between two regions (PP, PPI, EQ and their inverses); there are no coordinates anywhere, which is what makes the reasoning qualitative. The lower pane shows the internals of a node label — the expanded and unexpanded concept sets, the ANDS, ORS, SOMES, ALLS and number restrictions — i.e. the tableau state of that node, inspectable while the prover runs.

- an **edge atom** (a, b, w) with w from an edge query language $\mathcal{L}_{QE}$, where $\Phi(w)[x, y]$ has two free variables;
- an **equality atom** $(a, b, =)$, used chiefly as an implementation aid.

The semantics of an atom is entailment: a node i matches (x, C) if the substrate entails $\Phi(C)$ with x bound to i . For a propositional substrate that reduces to propositional deduction; for a DL substrate it is a theorem-proving request. This is the hinge of the whole design — *each query atom is an entailment question, not a lookup* — and it is what makes the query engine and the reasoner inseparable. It is also what the implementation’s dispatch mechanism rests on; see *Multimethods*, below.

Complex queries are then defined by an *inductive* grammar. A query is $\text{answer}(\vec{x}) \leftarrow k$, and a body k is either an atom or one of four constructs over bodies $k_1, \dots, k_n$:

$\neg(k_i)$	negation as failure
$(k_1 \wedge \dots \wedge k_n)$	conjunction
$(k_1 \vee \dots \vee k_n)$	disjunction
$\pi(\vec{x}) k_1$	projection, where every object of $\vec{x}$ occurs in k_1

Because the grammar is inductive, the operators apply to *arbitrary sub-bodies* rather than only to atoms. That is the substantive point: π is a compositional projection operator producing a new relation from a sub-body — a subquery — and $\neg$ applies to a body, so $\neg(\pi(\vec{x}) k)$ is a *negated projected subquery*. The remaining machinery follows: $\text{objects}(k)$ and $\text{vars}(k)$, the extension of Φ to bodies, the semantics of a query, and a $\models_{NAF}$ relation.

Two negations, deliberately kept apart. The thesis is careful that negation-as-failure is *not* the negation already available inside an atom. Even when the node query language offers it — $(a, \neg C)$ in an $\mathcal{ALC}$ instance query — the complement $V \setminus (a, C)^\varepsilon$ and the extension $(a, \neg C)^\varepsilon$ do not coincide, because substrates make no closed-world assumption. One is *not provably* C ; the other is *provably not* C . A query language over an open-world knowledge base needs both, and needs the boundary between them visible rather than conflated. Few query languages of the period drew it explicitly.

What SPARQL and SHACL reached later. Subqueries, negated subqueries, projection operators and the mixing of the two negations were all present in this grammar in 2005 and implemented in nRQL. SPARQL 1.0 became a Recommendation in January 2008 with neither subqueries nor negation; both arrived with SPARQL 1.1 in March 2013, as NOT EXISTS, MINUS and nested SELECT. SHACL’s constraint patterns cover much of the same ground again — a coding of the W3C SHACL Core test suite finds 73.5% of its constraints have a direct nRQL counterpart.

The distinctive part is not that the individual features appeared early. It is that they were defined over *one inductive grammar*, so they compose: any body can be projected, and any projection can be negated. Feature lists converge; a grammar that makes them compose is a design.

Considerable work then goes into **semantics-preserving transformations**: negation normal form, disjunctive normal form, the clause set $\mathcal{M}(q)$, arity unification across disjuncts, and individual substitution. These are what make the optimiser of Chapter 5 possible.

Two formal semantics, deliberately. The query language is then given a *second*, independent formal semantics, and the pair is the point.

The first is **logic-based**: the extension of a query is defined through entailment, by translating bodies into first-order formulae via Φ and asking what the substrate entails. It says *what the correct answer is*, and says it in the same currency as the substrate's own semantics and the description logic underneath. A logic-programming reading is given alongside it.

The second is **algebraic**. The extension of a body is constructed directly by set operations over the extensions of its sub-bodies: $\wedge$, $\vee$ and $\setminus$ become $\cap$, $\cup$ and $\setminus$ on sets of n -ary tuples. Because those operations need uniform arity, two further operations are defined — *projection* to reduce dimensionality and its inverse, *cylindrification*, to raise it — together with diagonals and the 1-vector. That is the vocabulary of cylindric algebra, and the justification offered is the known equivalence of non-recursive negation-free Datalog with relational algebra.

The division of labour is what matters. The logical semantics is the *specification*: it fixes what an answer means, and it is what lets query subsumption and satisfiability be posed as inference problems at all. The algebraic semantics is *constructive* — the thesis notes it is correspondingly implementation-near — and it is essentially the blueprint the query engine of Chapter 5 follows, which is why that engine's optimiser reasons about generators, testers and cross products rather than about entailment.

The two are presented as alternative accounts of the same language rather than proved equivalent; the argument for their agreement rests on the Datalog and relational-algebra correspondence. Giving both is unusual, and it is the reason the language can be specified logically and evaluated like a database query without the two descriptions drifting apart.

Inference problems for queries, and the QBox

Queries themselves become objects of inference: query satisfiability, unsatisfiable and tautological queries, and **query subsumption** and equivalence. From these comes the **QBox** — a directed acyclic graph whose nodes are queries and whose edges record immediate subsumption, always containing a $\top$ and a $\perp$ query.

The QBox is the query-level analogue of a TBox taxonomy, and it pays for itself twice: the stored results of *subsuming* queries serve as superset caches for candidate generators, and the stored results of *subsumed* queries can be returned immediately as subsets of the answer. Computing it requires a decision procedure for query subsumption, which Chapter 5 supplies for two of the four languages.

Finally **hybrid queries** are defined, spanning several substrates at once, with a notion of $*$ -consistent variable binding to tie the bindings together.

Chapter 5: the software realization

Chapter 5 (46,500 words) is the longest, and turns the formal model into Common Lisp. It divides into the data model realization, the query engine, and four concrete query languages.

Realizing the substrate

The object-oriented design starts from base classes and methods, then addresses three concerns. **Transient, persistent and virtual substrates** are handled through multiple inheritance, so persistence is a mixin rather than a separate hierarchy. **Physical representation independence** is achieved through iterators: provers and the query engine touch nodes and edges only through iterators and accessors, never through a concrete representation. And a family of **standard substrate classes** is supplied — the simple substrate, the data substrate, the disjunctive edge substrate, the RBox substrate, the JEPD substrate, the RCC substrates, description logic substrates, hybrid substrates and the geometric substrate — alongside standard description classes. Substrates for both guiding domains are then instantiated: the map substrates for the GIS, and RacerPro-backed substrates for Semantic Web retrieval.

Multimethods: choosing the reasoner per atom

One thread runs through the whole architecture and is easy to miss, because it lives in the programming language rather than the formalism. Chapter 4 defines the meaning of a query atom as *entailment*. Chapter 5 implements that entailment as a **binary generic function**, dispatched on the pair (*description or node type, query atom type*) — which single-dispatch object systems cannot express directly, and which CLOS gives for free.

In the sources, `matches-p` is specialised on exactly those pairs:

Query atom class	Description / node class
<code>substrate-simple-node-query</code>	<code>simple-node-description</code>
<code>substrate-racer-node-query</code>	<code>racer-node-description</code>
<code>substrate-predicate-node-query</code>	<code>substrate-node</code>
<i>... and the corresponding edge variants</i>	

Each delegates to `implies-p`, itself generic: its default method is a syntactic CNF implication test for data substrates, while a DL-backed description resolves instead to description logic entailment. Retrieval is selected the same way — `retrieve-matching-objects` carries a generic method on (*substrate, unary query*) and a specialised one on (*racer substrate, unary query*). Query subsumption uses a third such multimethod, `entails-p`, defined on $query \times query$.

Multiple inheritance then makes the class space a *product lattice* rather than a tree. Classes are formed by crossing a substrate family with a role:

data-substrate-node-query	= data-substrate-query × unary-query
data-substrate-edge-query	= data-substrate-query × binary-query
midelora-instance-retrieval-query	= midelora-abox-query × instance-retrieval-query
mirror-sql-data-substrate	= sql-data-substrate × mirror-data-substrate

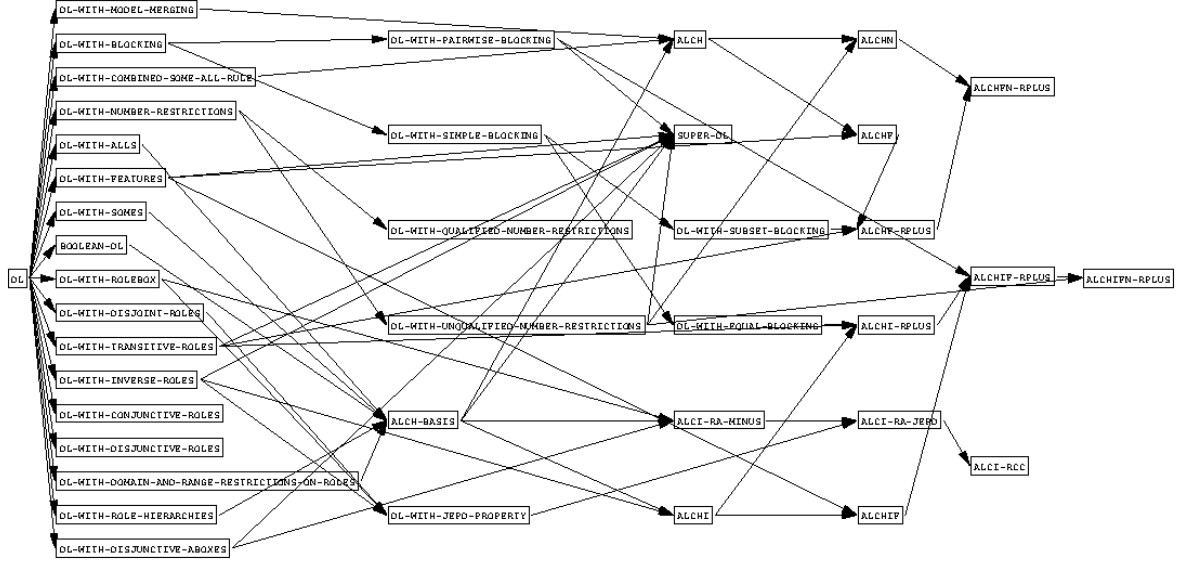


Figure 4: The *DL* axis of the MiDeLoRa space as an inheritance lattice. Feature mixins on the left (DL-WITH-TRANSITIVE-ROLES, DL-WITH-INVERSE-ROLES, DL-WITH-BLOCKING, ...) compose by multiple inheritance into the named logics on the right — *ALCH*, *ALCHN*, *ALCHIF_{R+}*, and the spatial *ALCI_{RCC}*. A prover is defined at whichever node is appropriate and inherits its rules along the lattice.

The consequence is the point. **Which entailment procedure runs is decided per atom, at runtime, by method resolution**, and behaviour can be defined at whichever level of the lattice fits — generically for all substrates, or specifically for one combination. A propositional substrate gets propositional deduction; a DL substrate a tableau prover; an RCC substrate composition-table reasoning; a geometric substrate a spatial index. Same query language, same atom syntax, different entailment relation, chosen by the pair.

There is a second speed to this. For *query atoms* the method is resolved at runtime, because the substrate a query touches is not known until it runs. For *tableau rules* the same lattice is resolved once, at composition time: a rule is conceptually a multimethod on (*language class*, *ABox class*), but what the dispatch returns is a *code template*, and the templates are composed into a prover before it is ever run. Selection by dispatch, execution by compiled code — the dispatch cost is paid once per prover rather than once per rule application.

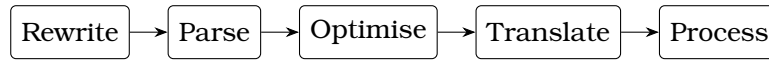
Entailment regimes, reached from the other side. SPARQL arrived at a comparable idea with *entailment regimes* — one query syntax, several entailment relations (RDF, RDFS, D-entailment, OWL 2 Direct and RDF-Based Semantics, RIF), so that basic graph pattern matching can be interpreted under a chosen semantics. That became a W3C Recommendation in March 2013, with the underlying work circulating in the working group for some years before.

The mechanism here differs in grain. A SPARQL entailment regime is fixed *per query or dataset*: one regime governs the whole evaluation. Here the method is selected *per atom, against the substrate that atom touches*, so a single hybrid query can have its concept atoms answered by a DL reasoner, its spatial atoms by a geometric index and its data atoms by a syntactic test — in one conjunction, under one semantics.

This is also the mechanism behind the thesis’s central claim. Covering *regions of the design space rather than points* is not a slogan about generality: the dispatch lattice is the region, and a new point in it is a new class plus the methods that distinguish it. The same construction reappears one layer down as the MiDeLoRa space — a prover chosen by *task* $\times$ *logic* $\times$ *ABox class* — applied to reasoners instead of atoms.

The query answering engine

Query processing is a seven-stage pipeline.



Optimisation (stage 4) operates on queries in normal form and has three parts: *semantic* optimisation, *determination of existential variables* — variables whose bindings need not be enumerated, only shown to exist — and *heuristic syntactic* optimisation.

The heuristic optimiser. This is the part that matters most, and it is a genuine query optimiser of the kind databases have, applied to DL ABox queries. Different variable binding orders for a disjunction-free, projection-free query give very differently performing **query execution plans**. A plan is the processing order of the conjuncts, written as a vector $\langle a_1, \dots, a_n \rangle$; a query with three conjuncts admits $3! = 6$ plans.

An atom’s position in the plan determines its **role**. A unary atom is either a *candidate generator* or a *tester* (filter). A binary atom takes one of four roles depending on how many of its objects are already bound: *generator*, *successor generator*, *predecessor generator*, or *tester*.

The consequence is concrete. For $(x, C) \wedge (y, D) \wedge (x, y, R)$, the plan $\langle (x, C), (x, y, R), (y, D) \rangle$ makes (x, C) a generator, (x, y, R) a successor generator and (y, D) a tester. The plan $\langle (x, C), (y, D), (x, y, R) \rangle$ instead makes both (x, C) and (y, D) generators, forcing a cross product of size $|(x, C)^{\mathcal{E}}| \times |(y, D)^{\mathcal{E}}|$ with (x, y, R) reduced to a filter. Every variable needs a generator; the optimiser’s job is to choose among the candidates by cost estimate.

Translation and processing. Stages 5–7 determine the query processing functions and generate them from **code templates** at compile time, so the cost of late binding is resolved once rather than per call. Function calls within those processing functions are minimised. Answer generators are *incremental and stream-based*, and *informed* — they consult index structures rather than enumerate blindly. The need for n -ary generators and testers is addressed, as is the use of index structures generally, support for hybrid substrate query languages, the query lifecycle and API, the concrete syntax, and the library of standard atoms (simple atoms, RacerPro description atoms, and λ -atoms).

The optimisations, and what each was worth

The thesis treats query optimisation as a first-class subject and — unusually — measures each technique separately, reporting the ones that disappointed as plainly as the ones that worked. The full set:

Semantic optimisation rewrites the query graph before planning, by two means. *Query classification* inserts the query into the QBox, so a subsuming query’s stored answers can serve as a superset cache for candidate generation and a subsumed query’s answers can be returned outright; a query found equivalent to $\top$ is flagged as tautological and one equivalent to $\perp$ as unsatisfiable, short-circuiting the evaluation entirely. *Query realization* — which the thesis states had not previously been investigated — is the query analogue of ABox realization: a conjunctive query is augmented with *implied atoms* so the search is better informed. Given $(x, \forall R.D) \wedge (x, C) \wedge (x, y, R)$, any y must be a D instance, but no atom says so, and a generator for y would enumerate $\top$ instances instead of D instances. Adding the implied (y, D) and $(y, \exists R^{-1}.C)$ cuts the branching factor. The thesis frames this explicitly in constraint-solving terms — it is *forward checking*, and what it avoids is *thrashing* — and is candid about the trade-off: more specific implied atoms make candidate generation itself more expensive, so the suggested criterion is to add only atoms an index can support.

Existential variables. A variable that appears in no other atom and not in the head need only be *witnessed*, not enumerated; generators stop after the first binding.

Heuristic plan optimisation. The conjunct ordering, as described above: each atom takes a role (generator, successor generator, predecessor generator, tester) determined by its position, and the optimiser costs the alternatives.

Code-template translation. Processing functions are generated by nesting code templates at compile time, so the whole query collapses into a single function for the root of the query graph — eliminating both the per-node `funcall` chain and graph traversal at answering time.

Informed, incremental generators, which consult index structures and stream results, plus efficient processing functions for role atoms, and n -ary generators and testers.

Technique	Measured	Verdict in the thesis
Heuristic plan optimisation	up to 10,000×	indispensable
Existential variables	1.013× typical 350× worst case	negligible in practice... ... but decisive on cross-product queries
Query classification (QBox)	2.1×	effective, and grows with query count
Code-template translation	1.568×	“not as effective as hoped”
Query realization	—	novel; not evaluated in time

Two of these deserve comment. The heuristic optimiser is not an optimisation but a precondition: without it, even simple queries over a single university department do not finish in acceptable time. And the QBox figure understates the mechanism, because it is measured over only four queries. The underlying numbers are striking — a DISKQL query over 2,694 map objects takes **155 s** cold; a more specific query that finds it as a parent then takes **17 s**, the next **0.38 s**, and a fourth, classified between two existing nodes, **0.72 s**. Without classification the same four cost 161, 123, 45 and 41 seconds. The saving compounds as the QBox fills.

On priority. These techniques were documented and implemented in 2004–05, and several were evaluated. Individual ingredients have longer lineages — subsumption-based caching goes back to work the thesis itself cites from 1994, forward checking to constraint solving, generator/tester plan ordering to relational query optimisation. What is specific here is the *combination*: this set of techniques assembled for conjunctive query answering over description logic ABoxes, implemented in a system with users, and measured technique by technique. That is a claim that rests on the dates and the numbers above, both of which are checkable.

Operational design: lifecycle, concurrency, incrementality

The requirements the thesis sets for the engine are operational rather than semantic, and the list reads more like a database query manager than a research prototype: incremental answering; **multi-user operation with queries processed concurrently; lifecycle management**, with queries as objects reachable and manipulable through an API; and resource-configurable answering — proactive or on-demand tuple computation, incomplete modes, and phased computation where cheap tuples are found first and expensive ones only on demand.

Three mechanisms deliver this.

Continuation-passing style. Processing functions are written so that each atom’s function calls a *continuation* to extend the current binding. That is what makes results streamable, suspendable and abortable partway through — and it is precisely what the code-template compiler optimises away, nesting the continuations into a single function at the query root so the `funccall` chain disappears at answering time.

A process pool. The sources carry a dedicated process pool, and the control surface is explicit: `prepare-to-run-query`, `query-running-p`, `abort-query`, `abort-process`,

`wait-for-request-or-abort` — with the same set for rules. Queries run in the background without blocking the caller, and can be cancelled while running.

Queries as first-class objects. A query is not a string submitted and forgotten. It is an object with a state, a lifecycle and an API: prepare it, start it, ask it for the next tuple, inspect what it is doing, abort it, or come back to it later. Incremental retrieval is a separate module in the sources (`get-next-tuple`).

Worth noticing in a 2005 reasoner. Incremental and abortable evaluation, concurrent execution of independent queries, non-blocking background processing, and a managed query lifecycle are not what description logic systems of the period typically offered — reasoners were usually synchronous, single-query, run-to-completion. Combining them with a reasoner-backed query semantics is the part that was unusual, and it is what made the two-phase and incomplete modes operationally meaningful: a mode that trades completeness for resources is only useful if the caller can also stop, resume, and ask for more.

Four query languages on one framework

Language	Substrate	Purpose
DSA	data substrate	queries over plain data values
BAA	ABox	generic conjunctive ABox query language
nRQL	ABox + data substrate	the Semantic Web / OWL retrieval language
DISKQL	spatial map substrate	spatial-thematic queries over the DISK

The **BAA** is the generic ABox query language. Its treatment includes efficient processing functions for role atoms, an algorithm for deciding **query subsumption**, and an algorithm for deciding query **satisfiability**, each worked through an example.

nRQL is the one that reached users, and it gets the most space. Refined requirements are derived first: concrete domain support, OWL-specific support, and modes for incomplete and two-phase processing. Its substrate is designed in two parts — an ABox substrate and a data substrate — and then the language itself: query concepts, **constraint atoms**, **head projection operators** for retrieving concrete-domain individuals and told values, and hybrid queries. Concrete syntax and worked examples follow, including the LUBM queries written in nRQL.

The operational design is where nRQL differs most from its contemporaries.

- **Four modes, 0–3**, of increasing completeness, with mode 3 complete. The incomplete modes work by enriching nRQL’s local cache and index structures with additional entries derivable by “simple” inference — described as fixpoint iterations that maximise told information. Crucially, *the original ABox is never modified*; the enrichment exists only in the caches. Mode 0 uses told information alone, splitting conjunctive concept assertions so that $i : C_1 \sqcap \dots \sqcap C_n$ yields each $i : C_i$. The incomplete modes reach acceptable completeness only for concept atoms whose query concept is *atomic*, since the enrichment processes cannot add arbitrary concept assertions.

- **Two-phase modes**, which return a cheap partial answer first and refine it, and **incremental** and concurrent answering.

Implementation covers the cache and index structures, the parser, the query syntax graph classes and the processing functions, heuristic refinement of the optimiser's cost estimates, the RacerPro support needed for two-phase operation, and a prototypical Semantic Web retrieval scenario with an OWL tree browser.

DISKQL and the **DLMAPS** system close the chapter: use of the map substrates, refined requirements, spatial node atoms and spatial edge atoms, the concrete syntax demonstrated on the DISKQL reference queries, and an algorithm for deciding subsumption between restricted DISKQL queries. The implementation covers the end-user syntax, the spatial atoms, adapted cost estimates for spatial edge atoms, and a *Query Inspector* GUI.

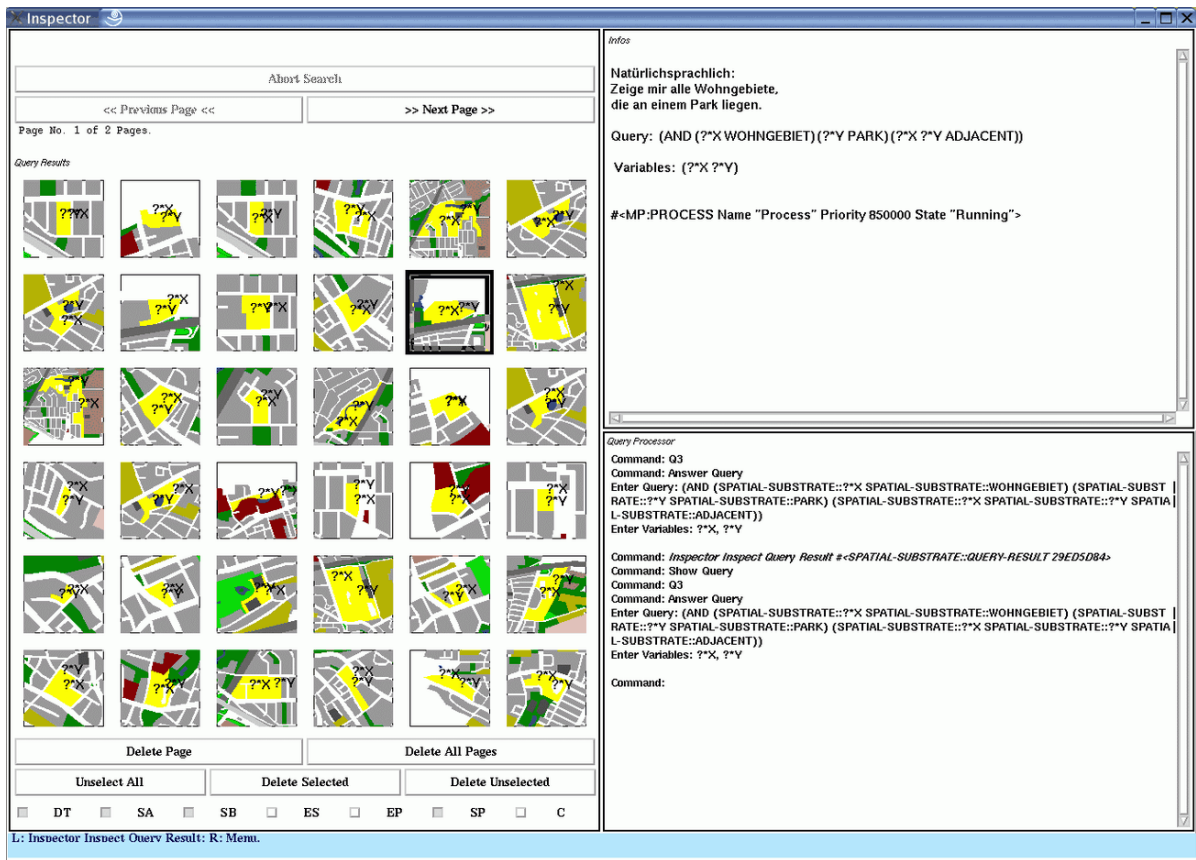


Figure 5: The DLMAPS *Query Inspector*, answering “show me all residential areas adjacent to a park” — in DISKQL, $(\text{AND } (?*X \text{ WOHNGBIET}) (?*Y \text{ PARK}) (?*X \text{ } ?*Y \text{ ADJACENT}))$. Each answer tuple is drawn as its own map crop with the bindings of $?*X$ and $?*Y$ highlighted, so a result set is inspected visually rather than read as a table. Note the *Abort Search* button and the page counter: results arrive incrementally and the search can be stopped while running, which is the operational design of the engine made visible.

Chapter 6: MiDeLoRa — the unpublished chapter

This chapter is not published anywhere else. The thesis states it plainly: the work in Chapter 6 was never published, only mentioned in passing in the *Journal of Applied Logic* article. At 36,200 words it is a quarter of the dissertation, and it is the part an English reader cannot otherwise reach.

MiDeLoRa (*Michael's Description Logic Reasoner*) is not primarily a reasoner. It is a **construction kit for building reasoners** — open, extensible, language- and task-specific DL provers. Throughout, a *prover* means the implementation of an inference procedure: it may implement a tableau calculus for a decision problem, or a higher-level service such as `concept_instances`, which will in turn call other provers.

Why build it at all

The thesis is careful to justify a new DL system when commercial ones existed. Four motivations are given, all rooted in the guiding domains: special DLs need special provers; task-specific provers can use optimisations a general ABox satisfiability prover cannot; existing provers sometimes need *internal modification*, such as the specialised $\exists$ -rule for RCC domain closure, which an external closed system cannot offer; and ABox representation needs to be open and extensible, which leads the chapter to question the standard Newell-style split between knowledge level and symbol level.

Provers as points in a space

A MiDeLoRa prover is a point in a three-dimensional space $T \times DL \times K$: the **task** it solves (ABox consistency, instance retrieval, role successors), the **description logic** it supports, and the **class of ABox** it operates on. TBox services such as taxonomy computation are deliberately excluded, on the grounds that the taxonomy algorithm and its optimisations are DL-independent; those are supplied as fixed template methods instead. You do not write a reasoner; you name a coordinate and assemble one.

An ABox is a substrate; the tableau is the ABox

The central design move unifies two things reasoners normally keep apart. The knowledge base and the tableau working data structure become **one object** — the same substrate instance from Chapter 4. The class is designed so one instance serves as both input ABox and tableau working structure, with the graph managed through the substrate framework and accessed only through iterators.

This forces a distinction between *old* individuals (original ABox individuals) and *new* ones created during expansion. From the input $\{i : \exists R.C\}$ any completion contains a

new individual: $\{i : \exists R.C, (i, temp_1) : R, temp_1 : C\}$. The distinction is needed to steer the expansion strategy, and because old and new individuals can differ in *quality* — in a spatial ABox the original individuals are instances of spatial datatypes in a spatial index, while a newly created individual’s geometry is generally unspecified. Accordingly only old individuals inherit from the geometric node class.

The payoff is that auxiliary information lives *on the object*. Finding every concept assertion about individual i is $O(1)$ rather than $O(n)$, and no separate bookkeeping structures must be kept consistent alongside the tableau — which the thesis notes is error-prone, memory-hungry and bad for maintainability.

The command history

Because rule application destructively mutates the tableau, restoring an earlier state requires *compensating operations*. Working on copies of parts of the object graph was tried and, in the author’s words, quickly unmasked as hopelessly inefficient in both time and memory.

MiDeLoRa therefore applies the **Command pattern**: every ABox-modifying operation is logged as a lightweight command object in a doubly linked history, each able to undo itself, with the requirement that *all* state changes are recorded. Restoring an earlier state means walking the history backwards from the current action, applying each command’s `undo`.

Not only is the tableau explicitly represented as a data structure — so is the proof process itself. The command history is a complete, explicit, navigable representation of the proof so far.

What that unlocks

The thesis argues this architecture enables five things conventional reasoners cannot easily do, and presents them as relevant to “next-generation” DL systems:

1. **Explanation and debugging** — the proof is inspectable rather than implicit in a call stack.
2. **True dependency-directed backtracking (DDB)**. Contemporary reasoners use the weaker dependency-directed *backjumping* (DDJ). Both DDJ and chronological backtracking restore the tableau state at a branch point, which also undoes expansion work that had nothing to do with the clash and may have to be redone — which is why caches matter so much for DDJ.

MiDeLoRa adds a second, *causal* chain between command objects: *precondition-for-actions*, with an inverse *postcondition-of-actions* for navigation in both directions. On a clash, the responsible branch points are located, their command objects marked, the causal chains traversed to their ends, and a second pass unmarks any command having an unmarked causal successor. The remaining marked commands are undone in chronological order and removed from the

history. Work performed after the culprit but not causally dependent on it *survives* — the bad parts are cut out of the tableau selectively.

3. **Persistence and recovery.** A database-backed ABox can be made persistent at any point. Checkpointing every n operations and logging the commands since is exactly a database write-ahead log, and the thesis makes the analogy explicit, citing log-buffer architectures. A proof can in principle survive a crash.
4. **Transactional reasoners.** Consistency and Isolation are reachable with coarse-grained locking; Durability and Atomicity need precisely the recovery and log-buffer architecture above. The author’s claim is blunt: DL systems have a long way to go here, and properties like transactionality can only be realised through explicit tableau representation and persistent command histories. The *computation state* of a DL system must be taken far more seriously than it has been.
5. **Proof migration** — moving a running proof to another machine for availability, or for multi-agent settings.

The chapter is candid that these are mostly *enabled* rather than realised, and the sources refine that usefully. The fine-grained causal machinery is unused: `precondition-for-actions` and its inverse are declared on `abox-item` but never written or read, and the two-phase marking algorithm has no implementation.

The *principle*, however, ships — by another route. Under **retention of deterministic assertions**, MiDeLoRa deliberately does not roll back to the start of a tableau expansion after a satisfiability test. Instead it removes only the assertions A with $\text{choice_points}(A) \neq \emptyset$ — those that depend on a choice — and keeps the deterministic remainder, which is logically implied and so remains valid. The next satisfiability test then skips re-expanding what survived. Chapter 7 measures the effect: the average ABox unsatisfiability test falls from **12 s to about 3.7 s**.

So dependency-directed retraction is real here, at the granularity of tableau expansions and keyed on choice-point sets. What stayed a sketch is the per-clash variant driven by the causal chain.

The chapter also asks the right sceptical question of itself: whether thoroughgoing OOP will cause performance and memory problems. It notes the mitigation — resolving dynamic dispatch at compile time where possible, as the query translator already did — and defers the answer to Chapter 7.

Base functionality, abstractions and rules

The remainder of the design covers what a construction kit must supply: TBox and ABox management; syntactic base functionality including computing which DL a given knowledge base actually uses; the TBox itself; and intensional standard inference services as **template methods**, fixed rather than overridable.

The structure of the MiDeLoRa space is then laid out, along with the software abstractions for defining provers, and an **arsenal of tableau rules** — standard rules, rules for non-standard DLs, and blocking conditions. State-of-the-art optimisations are surveyed and accommodated.

Implementation covers set representation, the command objects and history, provers and rule application chains, and the representation of models for the model merging test.

The rule definition language

The construction kit is driven by a **domain-specific language** for writing tableau rules — a Common Lisp macro language implementing a *five-port model*. It does not remove the need to program rules; it supplies the abstractions that make rules composable.

A rule is a point on the $DL \times K$ plane of the MiDeLoRa space, written as what is conceptually a CLOS multimethod on a language class and an ABox class:

```
(defrule or-expansion (dl abox) ...)
```

Rules therefore *inherit along both axes* by ordinary CLOS semantics — a rule for a more expressive logic inherits the implementation from a less expressive one unless it overrides it — and where the space’s structure does not give the right answer, a rule can request another’s implementation outright with `:inherit-from`. The two outcomes of a rule are marked in its body by `+insert-positive-code+` and `+insert-negative-code+`; the re-entry port is implicit in the return of control, and the backtracking port is `return-from prover`, which passes branch-point identifiers up to the parent node in the search tree.

Provers are then assembled from those chains by a second macro, `define-prover`, whose argument list is a coordinate in the MiDeLoRa space $T \times DL \times K$, and whose body names the three operational phases. An *ALCH* ABox satisfiability prover reads in full:

```
(define-prover ((abox-sat alch abox))
  (:init
    (perform (initial-abox-saturation)
      (:body
        (start-main))))
  (:main
    (perform (deterministic-expansion)
      (:body
        (if clashes
          (handle-clashes)
          (perform (or-expansion)
            (:positive
              (if clashes
                (handle-clashes)
                (restart-main)))
            (:negative
              (perform (some-expansion)
                (:positive
```

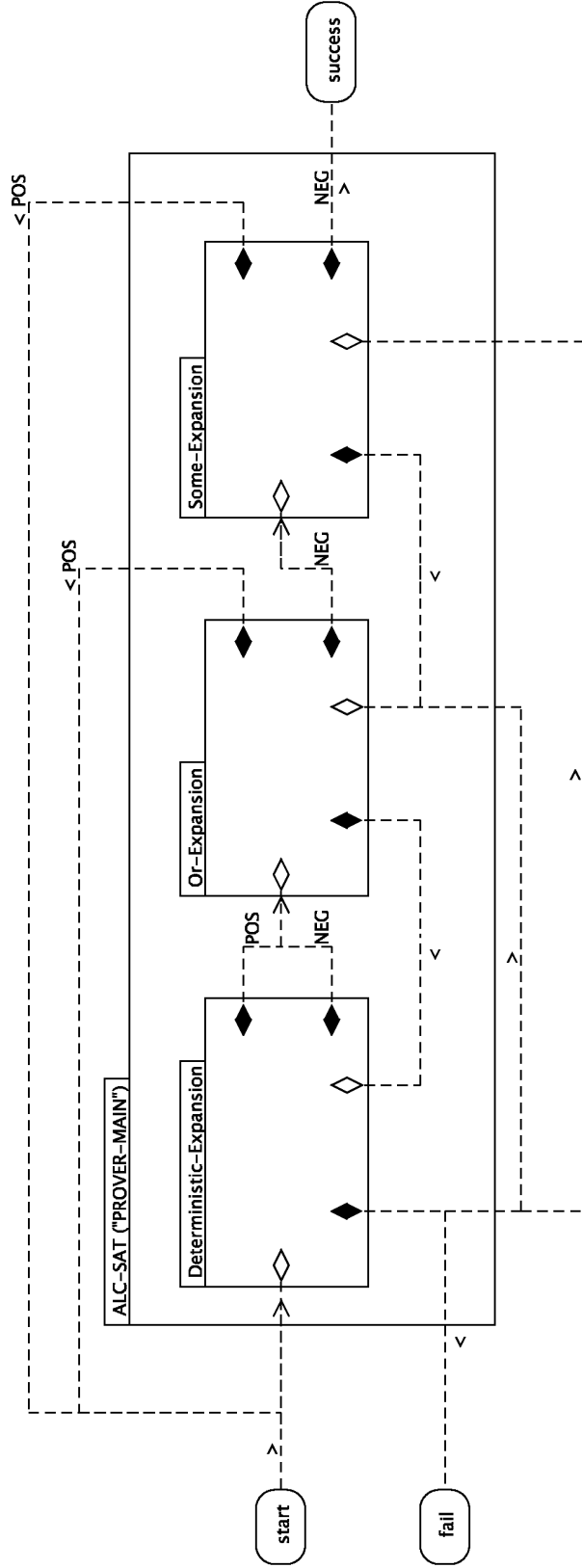


Figure 6: A complete prover as a wired rule application chain — the main phase of an *ALC* ABox satisfiability prover. Each rule is a box with the five ports: two entries (*main*, open diamond at left, and *re-enter*, open diamond at the bottom) and three exits (*positive* and *negative*, filled diamonds at right, and *backtrack*, filled diamond at the bottom). The strategy is the wiring: DETERMINISTIC-EXPANSION runs first, its NEG exit passes to OR-EXPANSION, whose NEG exit passes to SOME-EXPANSION, whose NEG exit is *success* — nothing left to expand. Each POS exit loops back to the top, creating the fresh incarnation a nondeterministic rule needs, and the backtrack exits run to *fail*. This is the same prover as the *define-prover* listing below, drawn.

```

      (if clashes
        (handle-clashes)
        (restart-main)))
    (:negative
      (success)))))))))
  (:success
    (completion-found)))

```

That is the whole prover. The tableau strategy is visible as structure rather than buried in control flow: `perform` fires a rule and dispatches on its ports, so `:positive` and `:negative` are literally the exits in the figure above. `restart-main` creates a fresh incarnation after a nondeterministic rule fired — the stack discipline the architecture requires — and a phase left undefined simply inherits by CLOS semantics, or can request another prover’s implementation with `:inherit-from`.

The performance decision is the interesting one. `defrule` does not define the rule directly — it defines a method returning a **code template**, and the rule application chain selects templates by $DL \times K$ at *composition time*. Late binding chooses the implementation once, when the prover is assembled, instead of on every rule application. Because rules are templates rather than functions, the compiler is then free either to emit a call to the next rule or to *inline its code* — “code unfolding” — though only when the next rule is deterministic, since a nondeterministic rule needs a fresh incarnation on the search stack to preserve its context.

One measurement that is missing. Whether compiling assembled prover definitions, rather than leaving them to be evaluated, actually pays is a natural question for this design, and the thesis does not answer it — no such comparison is reported. The nearest published figure is from the query side, where the same code-template technique was measured at $1.568\times$ and judged “not as effective as hoped”. Whether provers behave the same way is open; the rule-application path is much hotter than the query-translation path, so the result need not transfer.

Instantiations

The flexibility claim is discharged concretely: ABox satisfiability provers along an *ALCH* strand (*ALCH*, *ALCHf*, *ALCHf_{R+}*, *ALCHfN_{R+}*) and an *ALCHI* strand (*ALCHI*, *ALCHI_{fN_{R+}}*); an instance-retrieval prover; the BAA implemented for MiDeLoRa; provers for DLs with role boxes; a prover for **spatial** ABoxes; and a prover for **persistent** ABoxes.

Chapter 7: evaluation

The evaluation is notable for reporting against itself. All figures are on 2006 hardware (AMD XP 3000, 1 GB; a 4 GB 64-bit machine for the larger runs).

The query engine

Each base optimisation technique is measured separately: code-template-based translation, heuristic optimisation, special treatment of existential variables, and query classification. Effectiveness varies by query and substrate class.

The decisive finding concerns the optimiser. **Heuristic optimisation is not optional**: without it, even simple queries over a single university department do not finish in acceptable time, and the spread across conjunct orderings reaches a factor of **10,000**. Query classification, by contrast, helps only for certain query classes.

On scalability, all 14 LUBM queries are answered in **4.2 s at one university and 9.1 s at four** (78,579 individuals), scaling to 10 universities on the larger machine with an apparently linear curve. The thesis notes that relational database systems achieve no substantially better times on these queries, citing a comparative study the author supervised.

The DISK reference queries are harder. Either precomputing all RCC relations costs too much, or evaluating the virtual RCC atoms does; the substrate representation chosen — with or without explicit RCC edges — has a large effect. The verdict is that for a proof-of-concept implementation the times are adequate, and the framework can be used for such domains with good to sufficient query performance.

MiDeLoRa

The concept satisfiability tester and TBox reasoning are evaluated first, then the base ABox retrieval optimisations, then ABox scalability. For a full university (17,174 individuals):

Measure	Value
Load and index	152.5 s
Initial ABox satisfiability test	350.8 s
Initialisation query	38.1 s
All 14 LUBM queries	90.6 s
Instance tests incurred	68,711
decided obvious_instance	14,261 (20.76 %)
decided obvious_non_instance	54,435 (79.22 %)
ABox unsatisfiability tests actually needed	15 (0.02 %)
Global rollback time	20.2 s

On a logarithmic scale the curves are sub-linear, which argues for non-exponential growth. And the answer to Chapter 6's sceptical question is yes: a reasoner built on destructive mutation, explicit tableaux and a command history *can* be made efficient. The thesis notes that when work on MiDeLoRa began in 2003 it was not clear this was possible at all.

The honest negatives

The thesis does not oversell any of this, and three admissions are worth more than the positive results.

- **The successes are “fragile”** (*zerbrechlich*) and not transferable to arbitrary knowledge bases, queries or domains. They hold because 99.98% of instance tests are settled by *incomplete* methods. The residual 15 tests consume most of the 90.6 s, and whether the cheap paths hit is, in the author’s own words, largely luck. Even where the incomplete methods eliminate 99.12 % of tests — as for the DISK queries — the remaining 0.88 % still means 453 ABox unsatisfiability tests on a large ABox. The satisfiability test would need **one to two orders of magnitude** of speedup for robustness; binary or partition-based instance retrieval is identified as the promising route, and was not investigated in time.
- **Deep backtracking is expensive** — the architecture’s own bill. Rolling back a history of hundreds of thousands to millions of entries costs real time (20.2 s globally), so resetting to a branch point high in the search tree is inherently costly in MiDeLoRa. The thesis calls this “an important insight”, and it is: the same explicit history that enables DDB, persistence and migration is what makes deep backtracking dear.
- **RacerPro had already overtaken it.** The thesis says so directly, and goes further — asking whether the whole apparatus of incomplete and two-phase modes becomes pointless once reasoners get fast enough. Its answer is a qualified no: there will always be ABoxes where retrieval optimisations do not apply, and two-phase and incremental retrieval remain right for Semantic Web retrieval, where completeness of the answer set was never on offer anyway. The measured advantage of the incomplete modes was a factor of 10 on LUBM and 55 on the DISK reference queries, with lower memory use.

Closing claims

The concluding section states what the framework offers against the three problem areas, and lists the aspects it addresses that standard DL systems of the time did not: rich, virtual and persistent ABoxes; hybrid representations; incomplete and two-phase, resource-configurable query answering; incremental and concurrent query answering; expressive constructs for Semantic Web query languages (negation as failure, negated query roles, constraint atoms); and domain-specific languages for constructing and configuring provers. The author notes nRQL was, to his knowledge, the only ABox/Semantic Web query language in actual use with that feature set.

Future work is identified as persistence and transactionality, for which the interfaces and base abstractions already exist.

Where to read more

Most of the thesis is already reachable in English. Chapter 6 is the exception.

Chapter	Words	Where to find it in English
1 Introduction	5,100	the thesis's own English abstract
2 Foundations	29,100	standard background; see the <i>Description Logic Handbook</i>
3 Guiding domains	17,500	the DL workshop papers; spatial DL papers
4 Substrate model	11,700	summarised in the <i>Journal of Applied Logic</i> article
5 Realization, nRQL	46,500	summarised in the <i>Journal of Applied Logic</i> article
6 MiDeLoRa	36,200	nowhere — never published
7 Evaluation	9,200	partly in the JAL article and the scalability papers

The single most important pointer for an English reader: the *Journal of Applied Logic* article covers Chapters 3, 4, 5 and parts of 7. Chapter 6 does not appear in it beyond a mention.

The thesis also carries appendices worth knowing about: the LUBM TBox in KRSS syntax, the LUBM taxonomy, the two DISK maps, and a register of acronyms and abbreviations (A.5) that materially helps a reader of the German.

The dated record

Questions of precedence are best settled by dates, so here is the publication record underlying the thesis, taken from its own bibliography.

Year	Work	What it establishes
2000	<i>ALCRA</i> — <i>ALC</i> with Role Axioms; Decidable and Undecidable Extensions of <i>ALC</i>	the spatial DL line opens
2000–01	Obstacles on the Way to Spatial Reasoning with Description Logics	undecidability results
2002	On Spatial Reasoning with Description Logics	position paper
2003	Qualitative Spatial Reasoning with the <i>ALCI_{RCC}</i> -family	the decidable fragments
2003	Some Practical Issues in Building a Hybrid Deductive Geographic Information System	guiding domain 1
2004	Querying the Semantic Web with Racer + nRQL	nRQL in public
2004	Extended Query Facilities for Racer	the query facilities
2005	A High Performance Semantic Web Query Answering Engine	the engine and its optimiser
2006	On the Scalability of Description Logic Instance Retrieval	the retrieval optimisations
2006	A Flexible DL-based Architecture for Deductive Information Systems	the framework
2007	Design Principles for User Friendly, Interactive DL interfaces (OWLED)	the GUI work
2009	Flexible Software Architectures for Ontology-Based Information Systems, JAL 7(1), 75–99	the overview; accepted 2007

Set against that, the dissertation was **submitted in 2005**; **SPARQL 1.0** became a W3C Recommendation in **January 2008**; and **SPARQL 1.1**, with subqueries, NOT EXISTS, MINUS and the **entailment regimes** specification, in **March 2013**.

Read this record honestly, in both directions. The dates are what they are: nRQL was described in print in 2004 and its optimising engine in 2005, and the features SPARQL reached in 2013 were in it then. That is chronological precedence and it is checkable.

It is also worth saying that most of these are *workshop* papers — the Description Logic workshop, KRDB, OWLED — with one journal article at the end. Workshop proceedings are cited less, indexed less, and read less than journal or conference publications, which is one concrete reason this work is less visible than its dates would suggest. It is the sort of thing worth knowing about one’s own record rather than being surprised by.

And precedence is not influence. Similar ideas arrived later in other systems; whether they arrived independently, by way of this work, or through the shared background all of it drew on, the dates alone cannot say.

The code is public

The systems described in the thesis are on GitHub under <https://github.com/lambdamikel>, which most readers of the dissertation will not know:

- **MiDeLoRa** — the Chapter 6 reasoner framework
- **DLMAPS** — the Chapter 5 spatial system, described there as “the first hybrid ontology-based geographical information system that relies on Description Logic reasoning” and its query language as “an ancient predecessor of GeoSPARQL”
- **alcircc5** — *current* work on the open decidability question from Chapter 3: a Lean 4 machine-checked decision procedure for the $\forall$ PO-free fragment, with the full logic still open
- **shacl-nrql-comparison** — a retrospective analysis of nRQL against SHACL, SPARQL 1.1 and property graphs
- **OntoLisp, RacerPorter** — related Common Lisp semantic web tooling

nRQL anticipated much of SPARQL 1.1 — and of SHACL. nRQL (2004–05) combined reasoner-backed positive atoms with negation as failure, `project-to` as a compositional operator, union, aggregation, rules and concrete-domain predicates. SPARQL 1.0 reached Recommendation in **January 2008** without subqueries or negation; those arrived with **SPARQL 1.1 in March 2013**. The SHACL/nRQL comparison codes the 83 W3C SHACL Core constraint tests and finds **73.5 % have a direct nRQL counterpart**, 18.1 % a conditional one, and only **8.4 % are genuinely native to SHACL** — mostly unbounded property paths, which are not first-order definable and so provably out of nRQL’s reach.

That analysis is scrupulous about what this does and does not prove, and its distinction is worth preserving: chronological precedence is **strongly supported**, technical resemblance is **clear**, conceptual anticipation is **defensible** — but direct causal influence is **not established**. “Precedence is not influence.”

Two dates, both correct — and the earlier one matters. The dissertation was **submitted in 2005**. It was **defended on 14 April 2008**; the three-year gap was examination time, not work. (Logos Verlag catalogues the book edition as 2009, so both years appear in the wild — the dissertation is 2008, the book 2009.)

This is worth stating wherever the work is cited, because the **2005 submission date is the one that carries the precedence claims**. The content predates SPARQL 1.0’s Recommendation (January 2008) outright, and sits eight years before SPARQL 1.1 (March 2013). Citing only the 2008 publication date understates the priority by three years and places the work *after* SPARQL 1.0 rather than before it.

Did it deliver on its title?

The thesis set itself a programme in its own title: *flexible and configurable software architectures for data-intensive ontology-based information systems*. Taking the terms one at a time, and reading them as an engineer would rather than as a marketing department might:

Flexible — yes, and demonstrably. This is the best-supported claim in the work, because it was made falsifiable. The test was whether one framework could serve two domains chosen to pull in opposite directions: LUBM, which is large, shallow and conjunctive, and the DISK, which is small, deep and spatially hybrid. It served both. One data model was instantiated as description logic ABoxes, RDF graphs, RCC networks, data substrates and geometric substrates; four query languages were built on one framework; provers were assembled along two language strands plus task-specific ones. “Regions rather than points” turned out to be an operational statement about a dispatch lattice, not a slogan.

Configurable — yes. Configurability is shown the only way it can be, by exhibiting configurations that work: the rule definition language for building provers, the substrate and atom class lattice, and nRQL’s four modes plus its two-phase and incremental operation, which make the resource/completeness trade-off a runtime choice rather than a design-time one.

Software architectures — yes. Three frameworks that share one data model, an explicit comparison of layered, semi-integrated and integrated designs, and a reasoned commitment to the integrated one on cohesion grounds.

Data-intensive — partly, and the thesis says so. This is the weakest term and the one where the honest answer is qualified. LUBM at ten universities on 2006 hardware with apparently linear scaling is a real result. But MiDeLoRa in complete mode managed one university — 17,174 individuals — with a 350.8 s initial satisfiability test, the DISK needed the smaller maps, and the thesis describes its own successes as “fragile”, resting on incomplete methods that settle 99.98% of instance tests by luck as much as design. Its own verdict is that the satisfiability test needs one to two orders of magnitude of speedup before the results are robust. *Data-intensive* is achieved in the sense that scalability was addressed, measured, and shown to be non-exponential — not in the sense that these systems would stand next to a relational database.

That distinction deserves precision rather than modesty. DLMAPS and MiDeLoRa were research prototypes and are described as such. nRQL was not: it shipped inside a commercial reasoner, was used in production, and was the first Semantic Web repository to answer the LUBM queries completely. One component crossed into real use; the others demonstrated that the framework could produce them.

Overall. Three of the four terms are delivered convincingly and the fourth partially and honestly. A fair summary is that the thesis achieved the architectural programme it set itself and fell short of the performance programme — which is the ordinary outcome when a research prototype is measured against systems with orders of magnitude more engineering behind them, and is stated plainly in the evaluation rather than glossed.

The work’s most durable contribution is arguably not announced in the title at all. The reasoner construction kit of Chapter 6 — provers as points in a space, the tableau as a first-class inspectable object, the proof process itself represented as data — is the part that had no real counterpart then and still has few now.

Appendix: what holds up, and what doesn't

This section is a candid appraisal written for the author, not an orientation for a reader of the thesis. It is self-contained and can be removed without affecting anything above.

Holds up

The MiDeLoRa architecture is the most interesting thing here, and it is a road not taken. Making the proof process itself an explicit, navigable data structure — so that a reasoner can explain itself, backtrack by true dependency rather than by chronology, checkpoint like a database, and in principle migrate mid-proof — is an idea the DL community largely did not pursue. The argument that reasoners must take their *computation state* as seriously as databases do was right, and remains largely unaddressed. Explainability in particular is worth far more in 2026 than it was in 2008.

The negative result about backtracking cost is a real contribution. An explicit command history makes DDB, persistence and migration possible, and simultaneously makes deep backtracking expensive. Anyone attempting this architecture needs that finding, and it is recorded only here.

The substrate abstraction aged well — one labelled-graph model with pluggable label languages, spanning ABoxes, RDF, RCC networks and spatial data. The property-graph framing makes it legible to a modern audience, with the calibration above intact.

The query optimiser was ahead of its context. Generator/tester role assignment, cost-based plan selection, superset caching through the QBox, and code-template compilation are standard database technique applied to DL ABox queries at a time when that combination was rare. The measured factor of 10,000 between plans is the evidence.

The spatial DL thread is live, not historical. The decidability of $\mathcal{ALCI}_{RCC5}$, open since 2002–03, is still open. There is now a Lean 4 machine-checked decision procedure for the $\forall$ PO-free fragment with zero sorries, covering all four quadrants including the previously-open mixed PP/PO cases, with the full-logic route depending on an unproven width property. That is current research with a formal-methods component.

Doesn't

Chapter 2 is textbook background, competently done and entirely superseded. **The performance numbers** are 2006 hardware and of historical interest only — the thesis says as much, noting RacerPro had already overtaken MiDeLoRa. **nRQL as a language** lost to SPARQL, as the thesis half-predicted when it asked about the “half-life” of its own incomplete modes.

The thing actually worth noting

The DLMAPS README contains the real diagnosis: “Our implemented and documented techniques for geospatial, ontology-based query answering predate subsequent work on GeoSPARQL by years, yet there are almost no citations of our work.”

The SHACL comparison identifies why, without needing to allege exclusion: standards documents cite specifications rather than literature; the same idea circulated under four different vocabularies; system-specific syntax obscured the general algorithm; credit attaches to formalizations rather than engineering; and grounded semantics looked less general than model-theoretic ones.

That is a clear-eyed account of how good engineering work goes uncited — and it suggests the useful move is to make the specific anticipations legible and citable rather than to translate the book. That work is already underway in the GitHub repositories; it is mostly not yet discoverable from the thesis itself, and the thesis is not discoverable from it.

In short. There is real value here, and it is concentrated rather than spread: Chapter 6’s architecture, the backtracking trade-off, the query optimiser, and the live $\mathcal{ALCI}_{RCC5}$ problem. Chapter 2, the performance results and much of Chapter 7 are of historical interest. A reader with an hour should spend it on Chapter 6.